



Chapter 8. Automation with Nox

When you maintain a Python project, you're faced with many chores. Running checks on your code is an important part:

- Testing helps you reduce the defect rate of your code ([Chapter 6](#)).
- Coverage reporting pinpoints untested parts of your code ([Chapter 7](#)).
- Linters analyze your source code to find ways to improve it ([Chapter 9](#)).
- Code formatters lay out the source code in a readable way ([Chapter 9](#)).
- Type checkers verify the type correctness of your code ([Chapter 10](#)).

Other chores include:

- Building and publishing packages for distribution ([Chapter 3](#))
- Updating the dependencies of your project ([Chapter 4](#))
- Deploying your service (see [Example 5-7](#) in [Chapter 5](#))
- Building the documentation for your project

Automating these tasks has many benefits. You focus on coding while the check suite covers your back. You gain confidence in the steps that take your code from development to production. You remove human error and encode each process so others can review and improve it.

Automation gives you leverage to make each step as repeatable, each result as reproducible, as possible. Checks and tasks run in the same way on developer machines and in continuous integration (CI). They run across different Python versions, operating systems, and platforms.

In this chapter, you'll learn about Nox, a Python automation framework. Nox serves as a single entry point to your checks and tasks—for your

team, external contributors, and automated systems like CI runners.

You write Nox sessions in plain Python: each Nox session is a Python function that executes commands in a dedicated, isolated environment. Using Python as the automation language gives Nox great simplicity, portability, and expressivity.

First Steps with Nox

Install Nox globally using `pipx`:

```
$ pipx install --python=3.12 nox
```

Specify the latest stable version of Python—Nox defaults to that version when creating environments. `Pipx` makes the command `nox` available globally, while keeping its dependencies isolated from your global Python installation (see [“Installing Applications with pipx”](#)).

Configure Nox in a Python file named *noxfile.py* in your project, next to *pyproject.toml*. [Example 8-1](#) shows a *noxfile.py* for running the test suite. It’s meant for a simple Python project with no test dependencies besides `pytest`.

Example 8-1. A session for running the test suite

```
import nox

@nox.session
def tests(session):
    session.install(".", "pytest")
    session.run("pytest")
```

Sessions are the central concept in Nox: each session comprises an environment and some commands to run within it. You define a session by writing a Python function decorated with `@nox.session`. The function receives a session object as an argument, which you can use to install pack-

ages (`session.install`) and run commands (`session.run`) in the session environment.

You can try the session with the example project from previous chapters. For now, add your test dependencies to the `session.install` arguments:

```
session.install(".", "pytest", "pytest-httpserver", "factory-boy")
```

Invoke `nox` without arguments to run all sessions in the *noxfile.py*:

```
$ nox
nox > Running session tests
nox > Creating virtual environment (virtualenv) using python in .nox/tests
nox > python -m pip install . pytest
nox > pytest
===== tests session starts =====
...
===== 21 passed in 0.94s =====
nox > Session tests was successful.
```

As you can see from the output, Nox starts by creating a virtual environment for the `tests` session using `virtualenv`. If you're curious, you can find this environment under the *.nox* directory in your project.

NOTE

By default, environments use the same interpreter as Nox itself. In [“Working with Multiple Python Interpreters”](#), you'll learn how to run sessions on another interpreter, and even across multiple ones.

First, the session installs the project and `pytest` into its environment. The function `session.install` is just `pip install` underneath. You can pass any appropriate options and arguments to `pip`. For example, you can install your dependencies from a requirements file:

```
session.install("-r", "dev-requirements.txt")
session.install(".", "--no-deps")
```

If you keep your development dependencies in an extra, use the following:

```
session.install(".[tests]")
```

Above, you've used `session.install(".")` to install your project. Behind the scenes, pip builds a wheel using the build backend you've specified in *pyproject.toml*. Nox runs in the directory containing *noxfile.py*, so this command assumes both files are in the same directory.

Nox lets you use `uv` instead of virtualenv and pip for creating environments and installing packages. You can switch the backend to `uv` by setting an environment variable:

```
$ export NOX_DEFAULT_VENV_BACKEND=uv
```

Second, the session runs the `pytest` command you just installed. If a command fails, the session is marked as failed. By default, Nox continues with the next session, but it will exit with a nonzero status at the end if any session failed. In the run above, the test suite passes and Nox reports success.

[Example 8-2](#) adds a session to build packages for the project (see [Chapter 3](#)). The session also validates the packages using Twine's `check` command.

Example 8-2. A session for building packages

```
import shutil
from pathlib import Path

@nox.session
def build(session):
    session.install("build", "twine")

    distdir = Path("dist")
    if distdir.exists():
        shutil.rmtree(distdir)
```

```
session.run("python", "-m", "build")
session.run("twine", "check", *distdir.glob("*"))
```

[Example 8-2](#) relies on the standard library for clearing out stale packages and locating the freshly built ones: `Path.glob` matches files against wildcards, and `shutil.rmtree` removes a directory and its contents.

TIP

Nox doesn't implicitly run commands in a shell, unlike tools such as `make`. Shells differ widely between platforms, so they'd make Nox sessions less portable. For the same reason, avoid Unix utilities like `rm` or `find` in your sessions—use Python's standard library instead!

The programs you invoke with `session.run` should be available inside the environment. If they're not, Nox prints a friendly warning and falls back to the system-wide environment. In the Python world, running programs outside their intended environment is a mistake that's easy to make and hard to diagnose. Turn the warning into an error! [Example 8-3](#) shows how.

Example 8-3. Preventing external commands in Nox sessions

```
nox.options.error_on_external_run = True ❶
```

- ❶ Modify `nox.options` at the top of your *noxfile.py*, outside of sessions.

At times, you do need to run external commands, such as non-Python build tools. You can allow external commands by passing the `external` flag to `session.run`. [Example 8-4](#) shows how to build packages using an existing Poetry installation on your system.

Example 8-4. Using an external command to build the packages

```
@nox.session
def build(session):
```

```
session.install("twine")
session.run("poetry", "build", external=True)
session.run("twine", "check", *Path().glob("dist/*"))
```

You're trading off reliability for speed here. [Example 8-2](#) works with any build backend declared in *pyproject.toml* and installs it in an isolated environment on each run. [Example 8-4](#) assumes that contributors have a recent version of Poetry on their system and breaks if they don't. Prefer the first method unless every developer environment has a well-known Poetry version.

Working with Sessions

Over time, *noxfile.py* may accumulate a number of sessions. The `--list` option gives you a quick overview of them. If you add module and function docstrings with helpful descriptions, Nox includes them in the listing as well:

```
$ nox --list
```

```
Run the checks and tasks for this project.
```

```
Sessions defined in /path/to/noxfile.py:
```

```
* tests -> Run the test suite.
```

```
* build -> Build the package.
```

```
sessions marked with * are selected, sessions marked with - are skipped.
```

Running Nox with the `--session` option lets you select individual sessions by name:

```
$ nox --session tests
```

During development, running `nox` repeatedly lets you catch errors early. On the other hand, you don't need to validate your packages each time. Fortunately, you can change which sessions run by default by setting `nox.options.sessions`:

```
nox.options.sessions = ["tests"]
```

Now, when you run `nox` without arguments, only the `tests` session runs. You can still select the `build` session using the `--session` option. Command-line options override values specified in `nox.options` in *noxfile.py*.¹

TIP

Keep your default sessions aligned with the mandatory checks for your project. Contributors should be able to run `nox` without arguments to check if their code changes are acceptable.

Every time a session runs, Nox creates a fresh virtual environment and installs the dependencies. This is a good default, because it makes the checks strict, deterministic, and repeatable. You won't miss problems with your code due to stale packages in the session environment.

However, Nox gives you a choice. Setting up environments each time might be a tad slow if you re-run your tests in quick succession while coding. You can reuse environments with the option `-r` or `--reuse-existing-virtualenvs`. Additionally, you can skip installation commands by specifying `--no-install`, or combine these options using the shorthand `-R`:

```
$ nox -R
nox > Running session tests
nox > Re-using existing virtual environment at .nox/tests.
nox > pytest
...
nox > Session tests was successful.
```

Working with Multiple Python

Interpreters

If your project supports more than a single version of Python, you should run your tests on all of them. Nox really shines when it comes to running sessions on multiple interpreters. When you define the session with `@nox.session`, use the `python` keyword to request one or more specific Python versions, as shown in [Example 8-5](#).

Example 8-5. Running tests across multiple Python versions

```
@nox.session(python=["3.12", "3.11", "3.10"])
def tests(session):
    session.install(".[tests]")
    session.run("pytest")
```

Nox creates an environment for each version and runs the commands in those environments in turn:

```
$ nox
nox > Running session tests-3.12
nox > Creating virtual environment (virtualenv) using python3.12 ...
nox > python -m pip install '.[tests]'
nox > pytest
...
nox > Session tests-3.12 was successful.
nox > Running session tests-3.11
...
nox > Running session tests-3.10
...
nox > Ran multiple sessions:
nox > * tests-3.12: success
nox > * tests-3.11: success
nox > * tests-3.10: success
```


TIP

Did you get errors from pip when you ran Nox just now? Don't use the same compiled requirements file for every Python version. You need to lock dependencies separately for each environment (see [“Session Dependencies”](#)).

You can narrow sessions by Python version using the option `--python`:

```
$ nox --python 3.12
nox > Running session tests-3.12
...
```

During development, the `--python` option comes in handy, as it lets you save time by running tests on the latest version only.

Nox discovers interpreters by searching `PATH` for commands like `python3.12`, `python3.11`, and so on. You can also specify a string like `"pypy3.10"` to request the PyPy interpreter—any command that can be resolved against `PATH` will work. On Windows, Nox also queries the Python Launcher to find available interpreters.

Suppose you've installed a prerelease of Python and want to test your project on it. The `--python` option would require that the session lists the prerelease. Instead, you can specify `--force-python`: it overrides the interpreter for a single run. For example, the following invocation runs the `tests` session on Python 3.13:

```
$ nox --session tests --force-python 3.13
```

Session Arguments

So far, the `tests` session runs pytest without arguments:

```
session.run("pytest")
```

You *could* pass additional options—such as `--verbose`, which lists every individual test separately in the output:

```
session.run("pytest", "--verbose")
```

But you don't always want the same options for pytest. For example, the `--pdb` option launches the Python debugger on test failures. The debug prompt can be a lifesaver when you investigate a mysterious bug. But it's worse than useless in a CI context: it would hang forever since there's nobody to enter commands. Similarly, when you work on a feature, the `-k` option lets you run tests with a specific keyword in their name—but you wouldn't want to hardcode it in *noxfile.py* either.

Fortunately, Nox lets you pass additional command-line arguments for a session. The session can forward the session arguments to a command or evaluate them for its own purposes. Session arguments are available in the session as `session.posargs`. [Example 8-6](#) shows how you forward them to a command like pytest.

Example 8-6. Forwarding session arguments to pytest

```
@nox.session(python=["3.12", "3.11", "3.10"])
def tests(session):
    session.install(".[tests]")
    session.run("pytest", *session.posargs)
```

You must separate session arguments from Nox's own command-line arguments with the `--` delimiter:

```
$ nox --session tests -- --verbose
```

Automating Coverage

Coverage tools give you a sense of how much your tests exercise the code-base (see [Chapter 7](#)). In a nutshell, you install the `coverage` package and

invoke pytest via `coverage run`. [Example 8-7](#) shows how to automate this process with Nox.

Example 8-7. Running tests with code coverage

```
@nox.session(python=["3.12", "3.11", "3.10"])
def tests(session):
    session.install(".[tests]")
    session.run("coverage", "run", "-m", "pytest", *session.posargs)
```

When you're testing in multiple environments, you need to store the coverage data from each environment in a separate file (see [“Parallel Coverage”](#)). Add the following lines to *pyproject.toml* to enable this mode:

```
[tool.coverage.run]
parallel = true
```

In [Chapter 7](#), you installed your project in editable mode. The Nox session builds and installs a wheel of your project instead. This ensures that you're testing the final artifact you're distributing to users. But it also means that Coverage.py needs to map the installed files back to your source tree. Configure the mapping in *pyproject.toml*:

```
[tool.coverage.paths]
source = ["src", "*/site-packages"] ❶
```

- ❶ This maps files installed in the *site-packages* directory in an environment to files in your *src* directory. The key `source` is an arbitrary identifier; it's needed because you can have multiple mappings in this section.

[Example 8-8](#) aggregates the coverage files and displays the coverage report.

Example 8-8. Reporting the code coverage

```
@nox.session
def coverage(session):
    session.install("coverage[toml]")
    if any(Path().glob(".coverage.*")):
        session.run("coverage", "combine")
    session.run("coverage", "report")
```

The session invokes `coverage combine` only if there are any coverage data files—otherwise the command would fail with an error. As a result, you can safely use `nox -s coverage` to inspect your test coverage without having to re-run the tests first.

Unlike [Example 8-7](#), this session runs on the default Python version and installs only Coverage.py. You don't need to install your project to generate the coverage report.

If you run these sessions on the example project, make sure to configure Coverage.py as shown in [Chapter 7](#). Include Python 3.7 in the `tests` session if your project uses the conditional import for `importlib-metadata`:

```
$ nox --session coverage
nox > Running session coverage
nox > Creating virtual environment (uv) using python in .nox/coverage
nox > uv pip install 'coverage[toml]'
nox > coverage combine
nox > coverage report
```

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
src/.../__init__.py	29	2	8	0	95%	42-43
src/.../__main__.py	2	2	0	0	0%	1-3
tests/__init__.py	0	0	0	0	100%	
tests/test_main.py	36	0	6	0	100%	

TOTAL	67	4	14	0	95%	

```
Coverage failure: total of 95 is less than fail-under=100
nox > Command coverage report failed with exit code 2
nox > Session coverage failed.
```

NOTE

The `coverage` session still reports missing coverage for the `main` function and the `__main__` module. You'll take care of that in [“Automating Coverage in Subprocesses”](#).

Session Notification

As it stands, this *noxfile.py* has a subtle problem. Until you run the `coverage` session, your project will be littered with data files waiting to be processed. And if you haven't run the `tests` session recently, the data in those files may be stale—so your coverage report won't reflect the latest state of the codebase.

[Example 8-9](#) triggers the `coverage` session to run automatically after the test suite. Nox supports this with the `session.notify` method. If the notified session isn't already selected, it runs after the other sessions have completed.

Example 8-9. Triggering coverage reports from the tests

```
@nox.session(python=["3.12", "3.11", "3.10"])
def tests(session):
    session.install(".[tests]")
    try:
        session.run("coverage", "run", "-m", "pytest", *session.posargs)
    finally:
        session.notify("coverage")
```

The `try...finally` block ensures you get a coverage report even when a test fails. That's helpful when you start development with a failing test: you want to be sure the test exercises the code you're writing to make it pass.

Automating Coverage in Subprocesses

Alan Kay, a pioneer in object-oriented programming and graphical user interface design, once said, “Simple things should be simple; complex things should be possible.”² Many Nox sessions will be two-liners: a line to install dependencies and a line to run a command. Yet some automations require more complex logic, and Nox excels at those, too—primarily by staying out of your way and deferring to Python as a general-purpose programming language.

Let’s iterate on the `tests` session and measure coverage in subprocesses. As you saw in [Chapter 7](#), setting this up requires a little dance. First, you install a `.pth` file into the environment; this gives `coverage` a chance to initialize when the subprocess starts up. Second, you set an environment variable to point `coverage` to its configuration file. It’s tedious and a bit tricky to get right. Let’s “automate it away”!

First, you need to determine the location for the `.pth` file. The directory is named *site-packages*, but the exact path depends on your platform and the Python version. Instead of guessing, you can query the `sysconfig` module for it:

```
sysconfig.get_path("purelib")
```

If you called the function directly in your session, it would return a location in the environment where you’ve installed Nox. Instead, you need to query the interpreter *in the session environment*. You can do this by running `python` with `session.run`:

```
output = session.run(
    "python",
    "-c",
    "import sysconfig; print(sysconfig.get_path('purelib'))",
    silent=True,
)
```

The `silent` keyword lets you capture the output instead of echoing it to the terminal. Thanks to `pathlib` from the standard library, writing the `.pth` file now takes only a couple of statements:

```
purelib = Path(output.strip())
(purelib / "_coverage.pth").write_text(
    "import coverage; coverage.process_startup()"
)
```

[Example 8-10](#) extracts these statements into a helper function. The function takes a `session` argument, but it isn't a Nox session—it lacks the `@nox.session` decorator. In other words, the function won't run unless you call it from a session.

Example 8-10. Installing `_coverage.pth` into the environment

```
def install_coverage_pth(session):
    output = session.run(...) # see above
    purelib = Path(output.strip())
    (purelib / "_coverage.pth").write_text(...) # see above
```

You're almost done. What's left is invoking the helper function from the `tests` session and passing the environment variable to `coverage`.

[Example 8-11](#) shows the final session.

Example 8-11. The `tests` session with subprocess coverage enabled

```
@nox.session(python=["3.12", "3.11", "3.10"])
def tests(session):
    session.install(".[tests]") ❶
    install_coverage_pth(session)

    try:
        args = ["coverage", "run", "-m", "pytest", *session.posargs]
        session.run(*args, env={"COVERAGE_PROCESS_START": "pyproject.toml"})
    finally:
        session.notify("coverage")
```

Install the dependencies before the `.pth` file. The order matters because the `.pth` file imports the `coverage` package.

With subprocess coverage enabled, the end-to-end test produces the missing coverage data for the `main` function and the `__main__` module.

Invoke `nox` and watch it run your tests and generate a coverage report.

Here's what the report should look like:

```
$ nox --session coverage
nox > coverage report
Name                               Stmts   Miss Branch BrPart  Cover   Missing
-----
src/.../__init__.py                 29      0      8      0   100%
src/.../__main__.py                  2      0      0      0   100%
tests/__init__.py                    0      0      0      0   100%
tests/test_main.py                 36      0      6      0   100%
-----
TOTAL                               67      0     14      0   100%
nox > Session coverage was successful.
```

Parameterizing Sessions

The phrase “works for me” describes a common story: a user reports an issue with your code, but you can’t reproduce the bug in your environment. Runtime environments in the real world differ in a myriad of ways. Testing across Python versions covers one important variable. Another common cause of surprise is the packages that your project uses directly or indirectly—its dependency tree.

Nox offers a powerful technique for testing your project against different versions of a dependency. *Parameterization* allows you to add parameters to your session functions and supply predefined values for them; Nox runs the session with each of these values.

You declare the parameter and its values in a decorator named `@nox.parametrize`.³ [Example 8-12](#) demonstrates this feature and how it allows you to test against different versions of the Django web framework.

Example 8-12. Testing the project with multiple Django versions

```
@nox.session
@nox.parametrize("django", ["5.*", "4.*", "3.*"])
def tests(session, django):
    session.install(".", "pytest-django", f"django=={django}")
    session.run("pytest")
```

Parameterized sessions are similar to parameterized tests in pytest (see [Chapter 6](#)), from which Nox borrowed the concept. You can stack `@nox.parametrize` decorators to run sessions against all combinations of parameters:

```
@nox.session
@nox.parametrize("a", ["1.0", "0.9"])
@nox.parametrize("b", ["2.2", "2.1"])
def tests(session, a, b):
    print(a, b) # all combinations of a and b
```

If you want to check only for certain combinations, you can combine parameters in a single `@nox.parametrize` decorator:

```
@nox.session
@nox.parametrize(["a", "b"], [("1.0", "2.2"), ("0.9", "2.1")])
def tests(session, a, b):
    print(a, b) # only the combinations listed above
```

When running a session across Python versions, you're effectively parameterizing the session by the interpreter. In fact, Nox lets you write the following instead of passing the versions to `@nox.session`:⁴

```
@nox.session
@nox.parametrize("python", ["3.12", "3.11", "3.10"])
def tests(session):
    ...
```

This syntax is useful when you want specific combinations of Python and the dependency. Here's an example: as of this writing, Django 3.2 (LTE)

doesn't officially support Python versions newer than 3.10. Consequently, you need to exclude these combinations from the test matrix. [Example 8-13](#) shows how.

Example 8-13. Parameterizing by valid Python and Django combinations

```
@nox.session
@nox.parametrize(
    ["python", "django"],
    [
        (python, django)
        for python in ["3.12", "3.11", "3.10"]
        for django in ["3.2.*", "4.2.*"]
        if (python, django) not in [
            ("3.12", "3.2.*"), ("3.11", "3.2.*")
        ]
    ]
)
def tests(session, django):
    ...
```

Session Dependencies

If you followed [Chapter 4](#) closely, you may see some problems with the way Examples [8-8](#) and [8-11](#) install packages. Here are the relevant parts again:

```
@nox.session
def tests(session):
    session.install(".[tests]")
    ...

@nox.session
def coverage(session):
    session.install("coverage[toml]")
    ...
```

For one, the `coverage` session doesn't specify which version of `coverage` the project requires. The `tests` session does this right: it references the

tests extra in `pyproject.toml`, which includes the appropriate version specifiers (see [“Managing Test Dependencies”](#)).

The coverage session doesn’t need the project, though, so an extra seems like a bad fit. But before I get to this, let me point out another problem with the sessions above: they don’t lock their dependencies.

Running checks without locking dependencies has two drawbacks. First, the checks aren’t deterministic: subsequent runs of the same session may install different packages. Second, if a dependency breaks your project, checks fail until you exclude the release or another release fixes the problem.⁵ In other words, any project you depend on, even indirectly, has the power to block your entire CI pipeline.

On the other hand, lock file updates are a constant churn, and they clutter your Git history. Reducing their frequency comes at the price of running checks with stale dependencies. If you don’t require locking for other reasons, such as secure deployments—and you’re happy to quickly fix a build when an incompatible release wreaks havoc on your CI—you may prefer to keep your dependencies unlocked. There’s no such thing as a free lunch.

In [“Development Dependencies”](#), you grouped dependencies in extras and compiled requirements files from each. In this section, I’ll show you a lighter-weight method for locking: *constraints files*. You need only a single extra for it. It also doesn’t require installing the project itself, as extras usually do—which helps with the coverage session.

Constraints files look similar to requirements files: each line lists a package with a version specifier. Unlike requirements files, however, constraints files don’t cause pip to install a package—they only control which version pip selects *if* it needs to install the package.

A constraints file works great for locking session dependencies. You can share it across sessions while installing only the packages each session needs. Its only drawback, compared to using a set of requirements files, is that you need to resolve all dependencies together, so there’s a higher chance of dependency conflicts.

You can generate constraints files using pip-tools or uv (see [“Compiling Requirements with pip-tools and uv”](#)). Nox can automate this part, too, as shown in [Example 8-14](#).

Example 8-14. Locking the dependencies with uv

```
@nox.session(venv_backend="uv") ❶
def lock(session):
    session.run(
        "uv",
        "pip",
        "compile",
        "pyproject.toml",
        "--upgrade",
        "--quiet",
        "--all-extras",
        "--output-file=constraints.txt",
    )
```

- ❶ Explicitly require uv as the environment backend. (You could also install uv into the session with `session.install("uv")`, but uv supports a broader range of Python versions than you can use to install its PyPI package.)

The `--output-file` option specifies the conventional name for a constraints file, *constraints.txt*. The `--upgrade` option ensures that you get up-to-date dependencies whenever you run the session. The `--all-extras` option includes all the optional dependencies of the project.

TIP

Don't forget to commit the constraints file to source control. You need to share this file with every contributor, and it needs to be available in CI.

Pass the constraints file to `session.install` using the `--constraint` or `-c` option. [Example 8-15](#) shows the `tests` and `coverage` sessions with locked dependencies.

Example 8-15. Using a constraints file to lock session dependencies

```
@nox.session(python=["3.12", "3.11", "3.10"])
def tests(session):
    session.install("-c", "constraints.txt", ".[tests]")
    ...

@nox.session
def coverage(session):
    session.install("-c", "constraints.txt", "coverage[toml]")
    ...
```

Using a single constraints file requires that you target a well-known interpreter and platform. You can't use the same constraints file for different environments because each environment may require different packages.

If you support multiple Python versions, operating systems, or processor architectures, compile a separate constraints file for each environment. Keep the constraints files in a subdirectory to avoid clutter. [Example 8-16](#) shows a helper function that constructs filenames like *constraints/python3.12-linux-arm64.txt*.

Example 8-16. Constructing paths for a constraints file

```
import platform, sys
from pathlib import Path

def constraints(session):
    filename = f"python{session.python}-{sys.platform}-{platform.machine()}.txt"
    return Path("constraints") / filename
```

[Example 8-17](#) updates the `lock` session to generate the constraints files. The session now runs on every Python version. It uses the helper function to build the path for the constraints file, ensures that the target directory exists, and passes the filename to uv.

Example 8-17. Locking dependencies on multiple Python versions

```
@nox.session(python=["3.12", "3.11", "3.10"], venv_backend="uv")
def lock(session):
    filename = constraints(session)
    filename.parent.mkdir(exist_ok=True)
    session.run("uv", "pip", "compile", ..., f"--output-file={filename}")
```

The `tests` and `coverage` sessions can now reference the appropriate constraints file for each Python version. For this to work, you have to declare a Python version for the `coverage` session as well ([Example 8-18](#)).

Example 8-18. The `tests` and `coverage` session with multi-Python constraints

```
@nox.session(python=["3.12", "3.11", "3.10"])
def tests(session):
    session.install("-c", constraints(session), ".", "pytest", "coverage[toml]")
    ...

@nox.session(python="3.12")
def coverage(session):
    session.install("-c", constraints(session), "coverage[toml]")
    ...
```

Using Nox with Poetry Projects

If you manage your project with Poetry, you organize dependencies in dependency groups (see [“Dependency Groups”](#)). Dependency groups align naturally with Nox sessions: packages for the `tests` session go into a `tests` group, those for a `docs` session into a `docs` group, and so on. Using Poetry as the installer means you get locked dependencies for free—all installations honor the lock file.

Before I show you how to use Poetry in Nox sessions, let me call out a couple of differences between Poetry environments and Nox environments.

First, the Poetry environment is comprehensive: by default, it includes the project, its main dependencies, and every nonoptional dependency group. Nox environments only install packages required for the task they automate.

Second, the Poetry environment uses editable installs for the project so you don't need to reinstall after every code change. Nox environments build and install a wheel so automated checks see the project the same way an end user would.

There's no right and wrong here. Poetry environments are perfect for ad-hoc interactions with your project during development, with every tool just a `poetry run` away. Nox environments, on the other hand, are optimized for reliable and repeatable checks; they aim to be as isolated and deterministic as possible.

When you use Poetry in a Nox session, it's good to be mindful of these differences. I recommend these guidelines for invoking `poetry install` with Nox:

- Use the option `--no-root` to avoid an editable install of the project. If the session needs the project to be installed (not every session does), follow the command with `session.install(".")` to build and install a wheel.
- Use the option `--only=<group>` to install the appropriate dependency group for the session. If the session needs the project to be installed, list the special group `main` as well—this ensures that every package is pinned using *poetry.lock*.
- Use the option `--sync` to remove packages from a reused session environment when your project no longer depends on them.

[Example 8-19](#) puts this logic into a helper function that you can share across your session functions.

Example 8-19. Installing session dependencies with Poetry

```
def install(session, groups, root=True):  
    if root:
```

```

        groups = ["main", *groups]

    session.run_install(
        "poetry",
        "install",
        "--no-root",
        "--sync",
        f"--only={' '.join(groups)}",
        external=True,
    )
    if root:
        session.install(".")

```

The helper function uses `session.run_install` instead of `session.run`. The two functions work exactly alike, but `session.run_install` marks the command as an installation. This avoids package installations when you reuse environments with `--no-install` or `-R`.

If you've followed along using Poetry in Chapters [6](#) and [7](#), your *pyproject.toml* should have a dependency group named `tests` containing `pytest` and `coverage`. Let's split the `coverage` dependency off into a separate group, since you don't need the test dependencies for the `coverage` session:^{[6](#)}

```

[tool.poetry.group.coverage.dependencies]
coverage = {extras = ["toml"], version = ">=7.2.7"}

[tool.poetry.group.tests.dependencies]
pytest = ">=7.4.4"

```

Here's how you'd install the dependencies in the `tests` session:

```

@nox.session(python=["3.12", "3.11", "3.10"])
def tests(session):
    install(session, groups=["coverage", "tests"])
    ...

```

And here's what the `coverage` session looks like with the helper function:


```
@nox.session
def coverage(session):
    install(session, groups=["coverage"], root=False)
    ...
```

TIP

How does Poetry know to use a Nox environment instead of the Poetry environment? Poetry installs packages into the active environment, if one exists. When Nox runs Poetry, it activates the session environment by exporting the `VIRTUAL_ENV` environment variable (see [“Virtual Environments”](#)).

Locking Dependencies with nox-poetry

There’s an alternative approach to using Nox with Poetry projects. The `nox-poetry` package is an unofficial plugin for Nox that lets you write sessions without worrying about locking, using plain `session.install`. Behind the scenes, `nox-poetry` exports a constraints file for pip using `poetry export`.

Install `nox-poetry` into the same environment as Nox:

```
$ pipx inject nox nox-poetry
```

Decorate your sessions with `@session` from the `nox-poetry` package, which is a drop-in replacement for `@nox.session`:

```
from nox_poetry import session

@session
def tests(session):
    session.install(".", "coverage[toml]", "pytest")
    ...
```

When you install packages with `session.install`, the constraints file keeps their versions in sync with `poetry.lock`. You can manage depen-

dencies in separate dependency groups or put them into a single `dev` group.

While this approach is convenient and simplifies the resulting *noxfile.py*, it doesn't come for free. Translating Poetry's dependency information into a constraints file isn't lossless—for example, it doesn't include package hashes or the URLs of private package repositories. Another drawback is that contributors have another global dependency to worry about. When I wrote `nox-poetry` in 2020, dependency groups didn't exist. As of this writing, I recommend using Poetry directly, as described in the previous section.

Summary

Nox lets you automate checks and tasks for a project. Its Python configuration file *noxfile.py* organizes them into one or more sessions. Sessions are functions decorated with `@nox.session`. They receive a single argument `session` providing the session API ([Table 8-1](#)). Every session runs in an isolated virtual environment. If you pass a list of Python versions to `@nox.session`, Nox runs the session across all of them.

Table 8-1. The session object

Attribute	Description	Example
<code>run()</code>	Run a command	<code>session.run("coverage", "report")</code>
<code>install()</code>	Install packages with pip	<code>session.install(".", "pytest")</code>
<code>run_install()</code>	Run an installation command	<code>session.run_install("poetry", "install")</code>
<code>notify()</code>	Enqueue another session	<code>session.notify("coverage")</code>
<code>python</code>	The interpreter for this session	<code>"3.12"</code>
<code>posargs</code>	Extra command-line arguments	<code>nox -- --verbose --pdb</code>

The command `nox` ([Table 8-2](#)) provides a single entry point to your suite of checks. Without arguments, it runs every session defined in *noxfile.py* (or those you’ve listed in `nox.options.sessions`). The earlier you identify issues with your code, the cheaper it is to fix them—so use Nox to run the same checks locally as in continuous integration (CI). Besides checks, you can automate many other chores, such as building packages or documentation.

Table 8-2. Command-line options for `nox`

Option	Description	Example
<code>--list</code>	List the available sessions	<code>nox -l</code>
<code>--session</code>	Select sessions by name	<code>nox -s tests</code>
<code>--python</code>	Select sessions by interpreter	<code>nox -p 3.12</code>
<code>--force-python</code>	Override the interpreter for a session	<code>nox --force-python 3.13</code>
<code>--reuse-existing-virtualenvs</code>	Reuse existing virtual environments	<code>nox -rs tests</code>
<code>--no-install</code>	Skip installation commands	<code>nox -Rs tests</code>

There's a lot more to Nox that this chapter didn't cover. For example, you can use Conda or Mamba to create environments and install packages. You can organize sessions using keywords and tags, and assign friendly identifiers using `nox.param`. Last but not least, Nox comes with a GitHub Action that makes it easy to run Nox sessions in CI. Take a look at the [official documentation](#) to learn more.

- 1 In case you're wondering, always use the plural form `nox.options.sessions` in `noxfile.py`. On the command line, both `--session` and `--sessions` work. You can specify any number of sessions with these options.
- 2 Alan Kay, [“What Is the Story Behind Alan Kay’s Adage ‘Simple Things Should Be Simple, Complex Things Should Be Possible’?”](#), *Quora Answer*, June 19, 2020.
- 3 Like `pytest`, Nox uses the alternate spelling “parametrize” to protect your “E” key-cap from excessive wear.
- 4 The eagle-eyed reader may notice that `python` is not a function parameter here. If you do need it in the session function, use `session.python` instead.
- 5 Here, Semantic Versioning constraints harm more than help. Bugs occur in all releases, and your upstream's definition of a breaking change may be narrower

than you like. See Hynek Schlawack, [“Semantic Versioning Will Not Save You”](#), March 2, 2021.

- 6 Run `poetry lock --no-update` after editing *pyproject.toml* to update the *poetry.lock* file.